

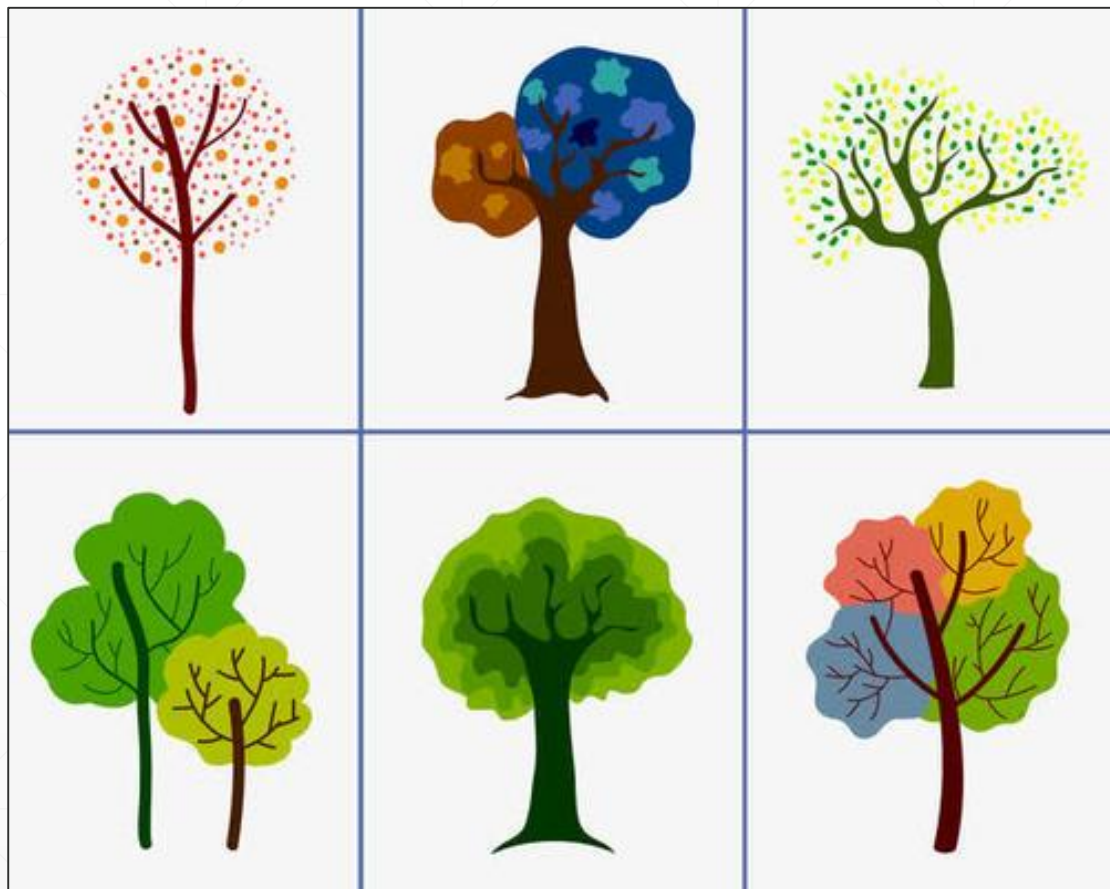
Set之一



Set基础

北京理工大学计算机学院
金旭亮

什么是Set?



Set是一种集合，其顺序无关紧要，最重要的特性，是它里面不允许有重复元素。

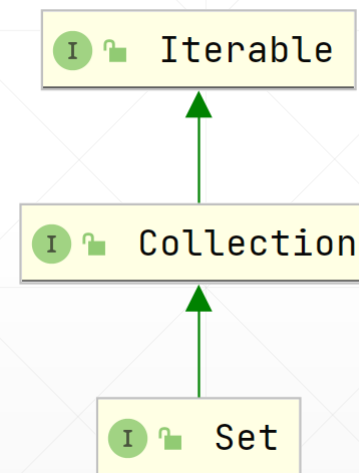
HashSet



JDK 中的 HashSet 组件，实现了 Set 接口，是最常用的 Set 类型。

```
private static void onlyOne() {  
    //创建一个包容字符串的Set集合  
    Set<String> books = new HashSet<>();  
    //添加一个字符串对象  
    books.add("Hello");  
    //再次添加一个字符串对象，  
    //因为两个字符串对象通过equals方法比较相等，所以添加失败，返回false  
    boolean result = books.add(new String(original: "Hello"));  
    System.out.println(result); //false  
    //下面输出看到集合只有一个元素  
    System.out.println(books); // [Hello]  
}
```

WhatIsSet.java



巧用Set基础特性移除重复的元素

//使用这种方法，可以轻松移除重复元素

```
private static void removeDuplicateElements() {  
    //Set中不允许有重复的元素  
    Set<String> stringSet = new HashSet<>();  
    String[] strings = {"a", "b", "c", "c"};  
    stringSet.addAll(Arrays.asList(strings));  
    //输出: [a, b, c]  
    System.out.println(stringSet);  
}
```

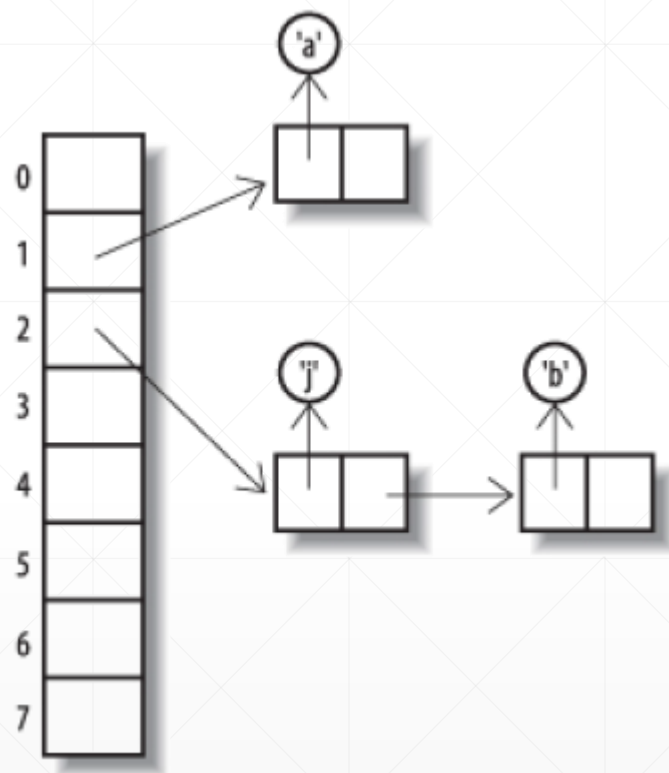
重要提示



由于Set需要保证集合中没有重复元素，因此，如果不能实现“常数级别”的搜索速度，那么，数据越多，向其中插入新元素的速度越慢，因为每插入一个新元素，Set都必须保证它以前没有被插入过。

理解HashSet的基本原理

- HashSet将对象的哈希码（hashCode）作为访问对象的“索引（index）”。
- 当向HashSet存入一个对象时，它会同时调用hashCode()和equals()两个方法确定此对象是否已在集合中，如果发现已经有了，则不会把此对象加入到集合中。
- HashSet中每个存储元素的“slot（槽）”通常被称为“bucket”，如果有多个元素的hashCode相等，但它们equals()方法返回false，这就导致同一个槽中放置多个元素，这会导致性能下降。



放在Set中的元素，它的hashCode()和equals()方法的返回值，起着关键性的作用。

重写equals和hashCode方法

Set通过调用对象的equals和hashCode方法确定两个对象是否相同，因此，需要**保证这两个方法返回一致的结果**。

当重写equals和hashCode()方法时，应该遵循以下规则：

1

在程序运行过程中，同一个对象多次调用hashCode()方法应该返回相同的值。

2

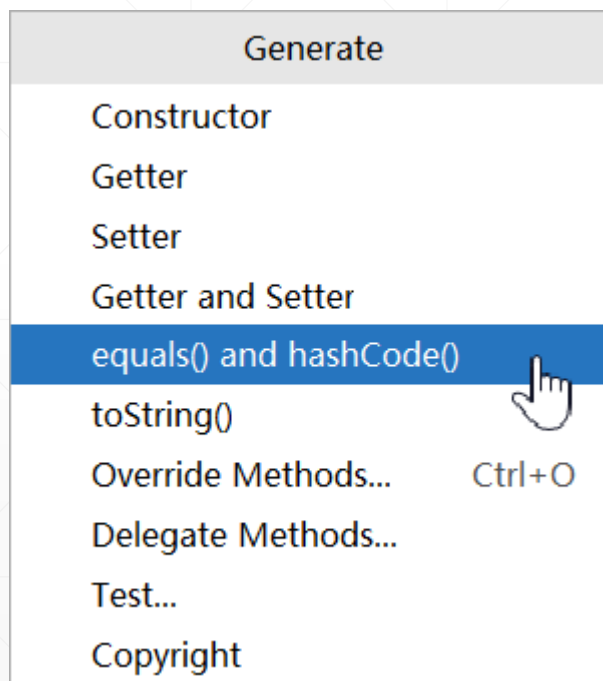
当两个对象通过equals()方法比较返回true时，这两个对象的hashCode()方法应返回相等的值。

3

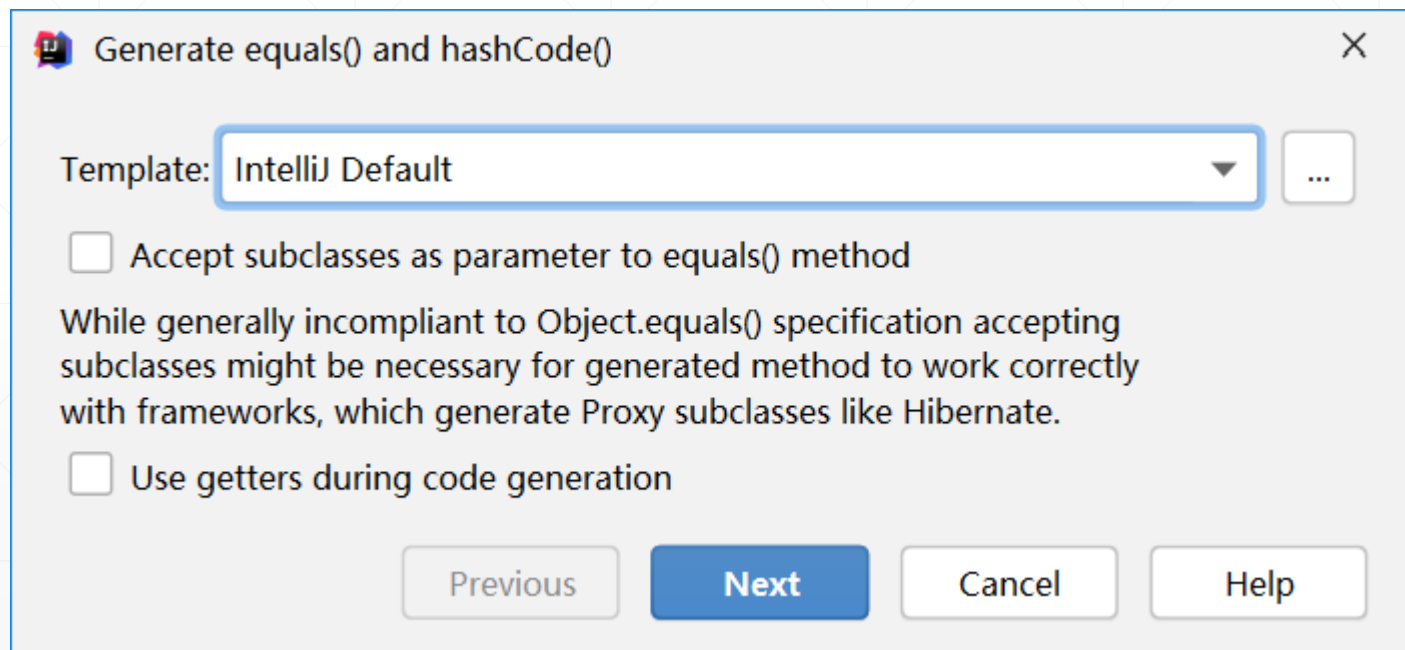
对象中用作equals()方法比较标准的实例变量，都应该用于计算hashCode值。

使用工具自动生成equals()和hashCode()方法代码

放到HashSet中的对象，通常需要定义它的equals()和hashCode()方法，IntelliJ可以帮助我们完成这一工作。



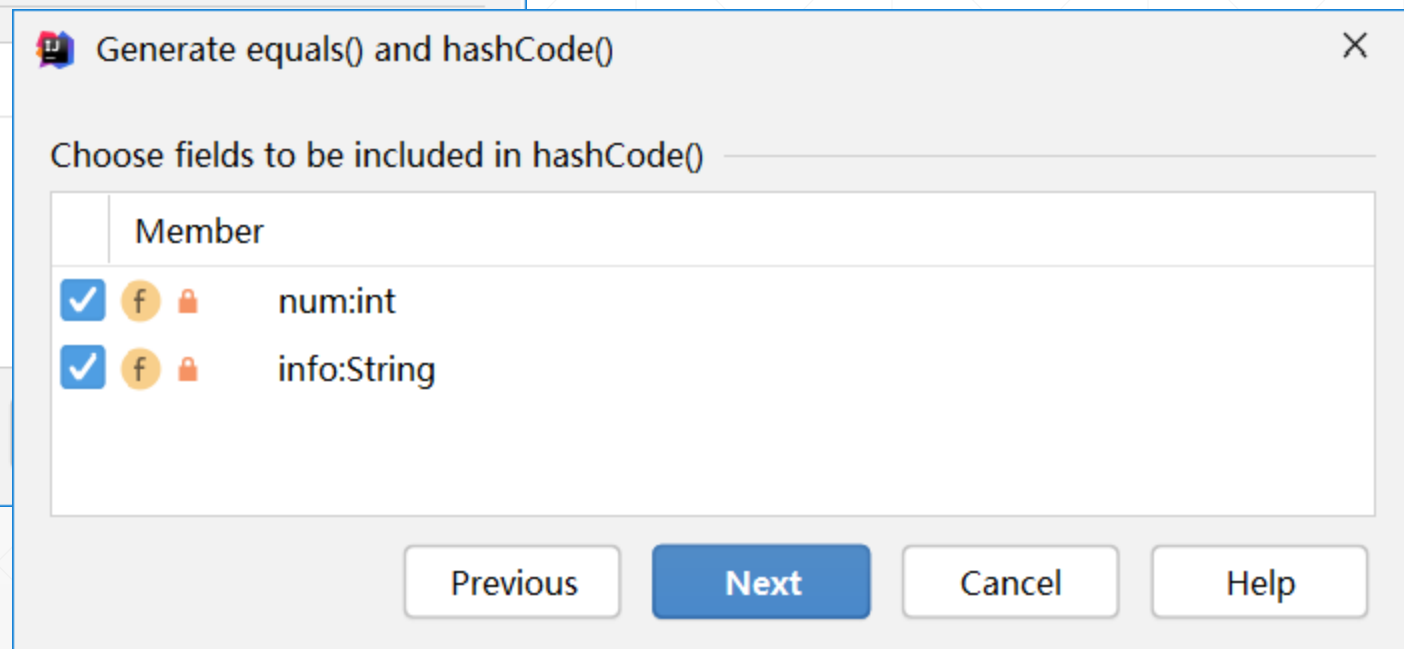
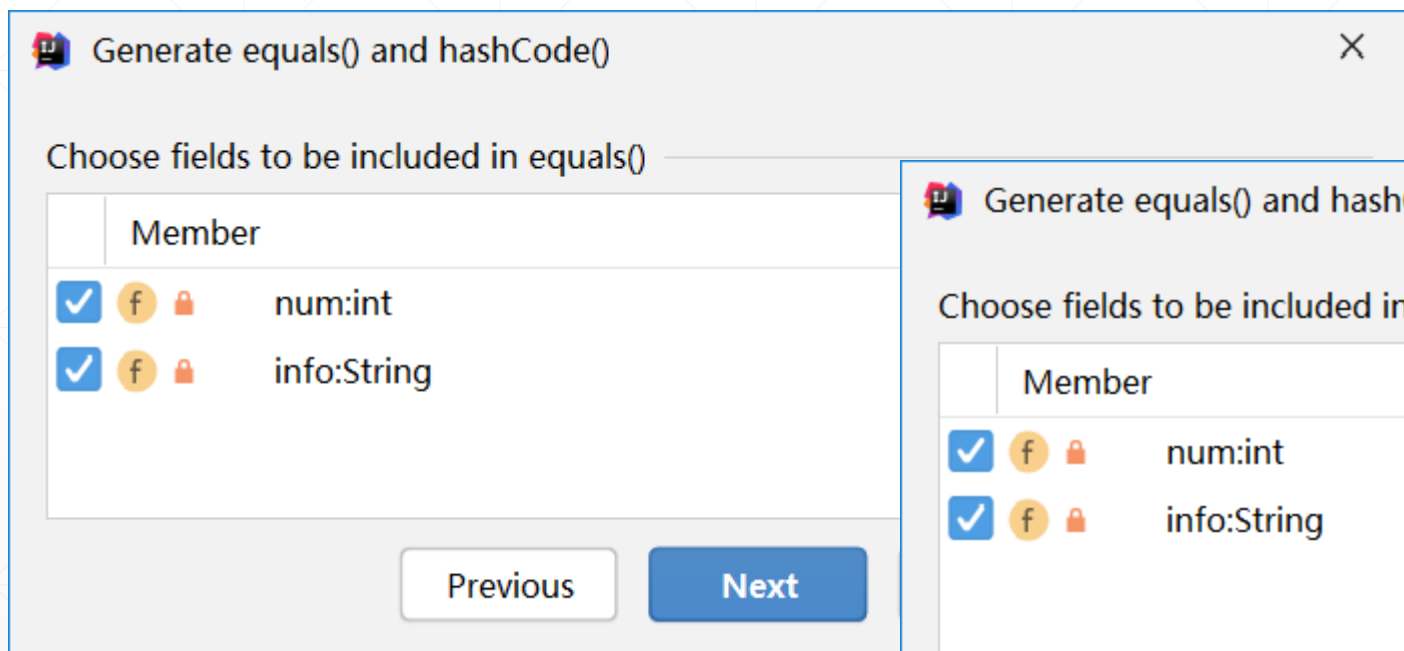
Alt+Ins组合键调出



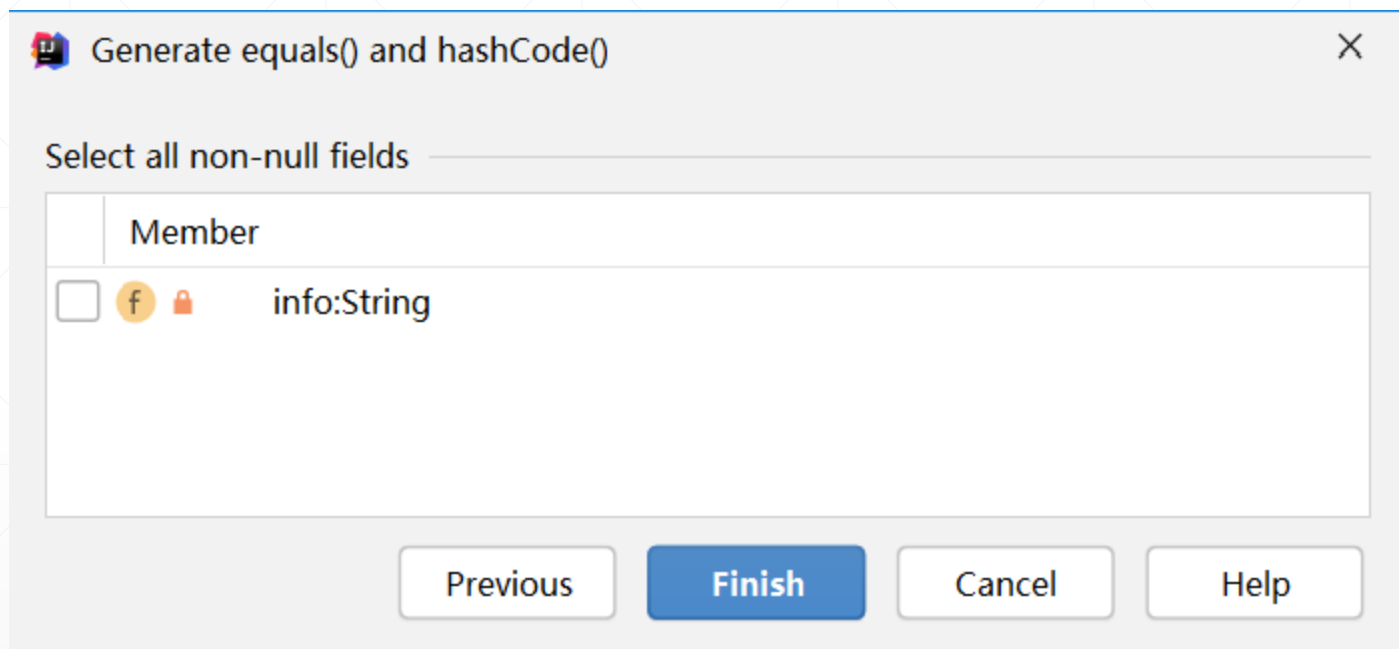
IntelliJ使用的代码模板有好几个，这里用的是其内置的默认代码模板。

使用IntelliJ生成代码

比如，MyClass类中有int nums和String info两个字段，则IntelliJ会启动一个向导帮助我们生成equals方法和hashCode方法。



指定哪个field是不能为null的



最终的结果

如果需要，你也可以用自己的算法替换掉IntelliJ生成的方法代码，但多数场景下，生成的代码足以满足需求。

```
public class MyClass {
    private int num;
    private String info;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;

        MyClass myClass = (MyClass) o;

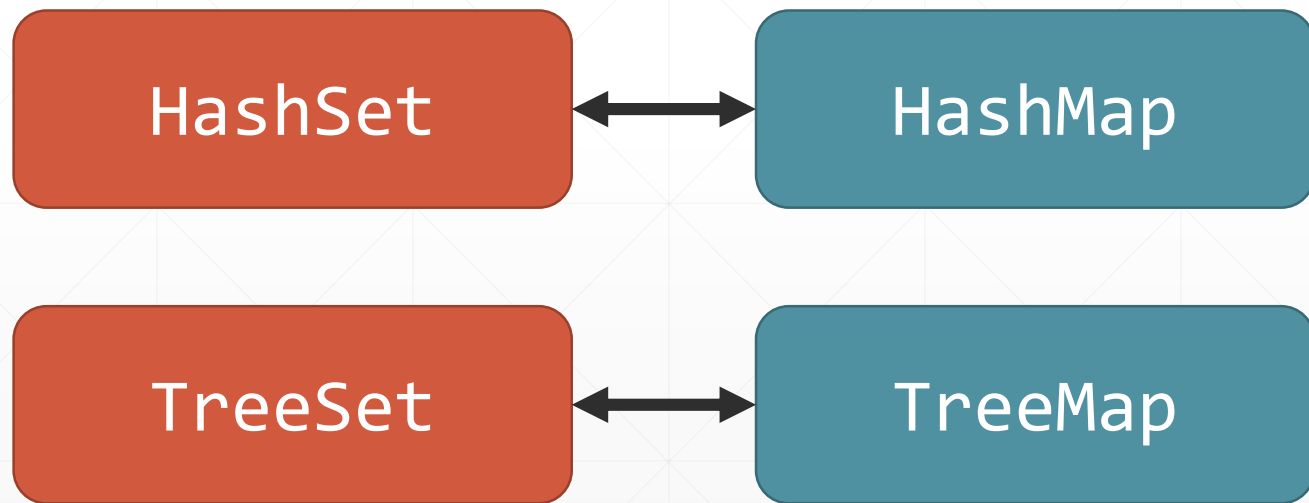
        if (num != myClass.num) return false;
        return info != null ? info.equals(myClass.info) :
            myClass.info == null;
    }

    @Override
    public int hashCode() {
        int result = num;
        result = 31 * result + (info != null ? info.hashCode() : 0);
        return result;
    }
}
```

Set如何存储数据?

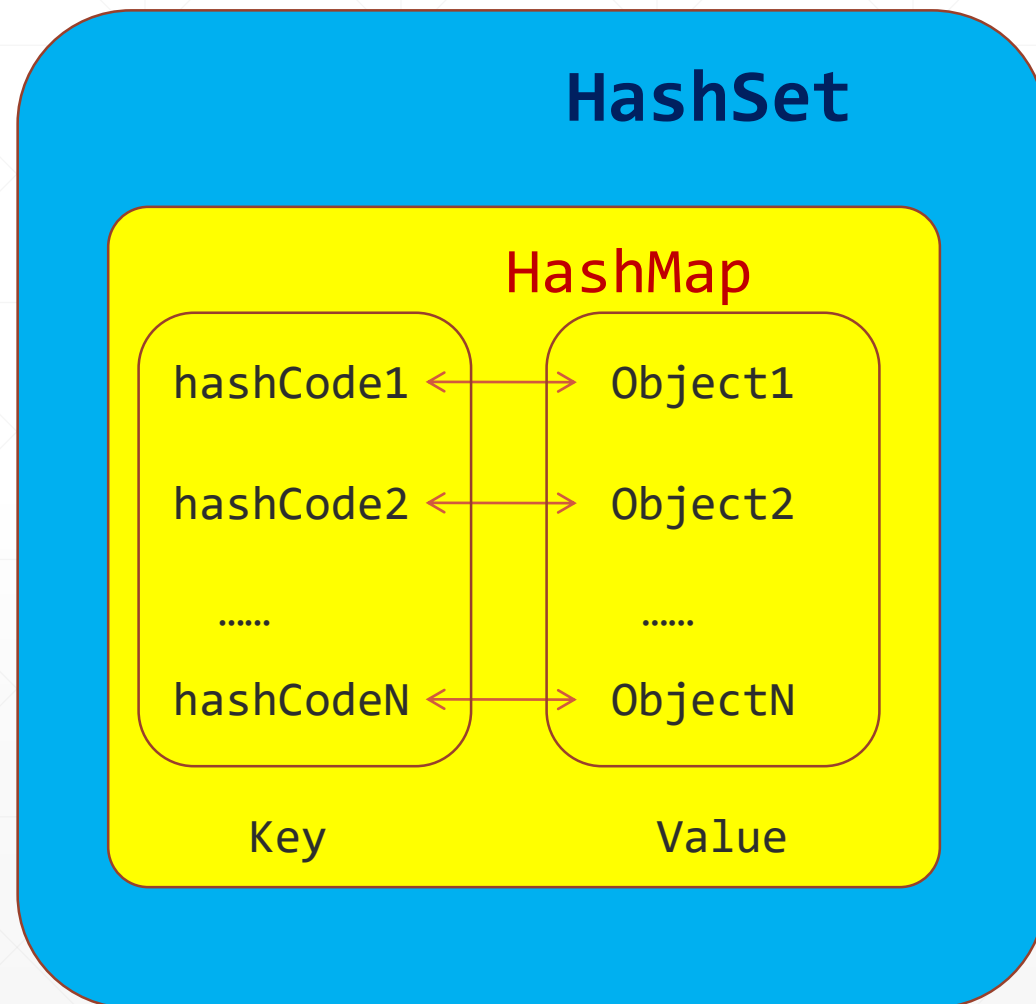


JDK中常用的Set有HashSet和TreeSet，它们都与对应的Map有着密切的联系，其内部使用Map存储数据。



HashSet技术内幕

HashSet在内部包容一个HashMap，
以对象的hashCode作为这一Map
的Key。



通过实例了解HashSet的特性

```
class R {  
    int value;  
    public R(int value) {  
        this.value = value;  
    }  
    public boolean equals(Object obj) {  
        if (obj instanceof R) {  
            R r = (R) obj;  
            return r.value == this.value;  
        }  
        return false;  
    }  
    public int hashCode() {  
        return this.value;  
    }  
  
    public String toString() {  
        return "R(value:" + value + ",hashCode:"  
            + this.hashCode() + ")";  
    }  
}
```

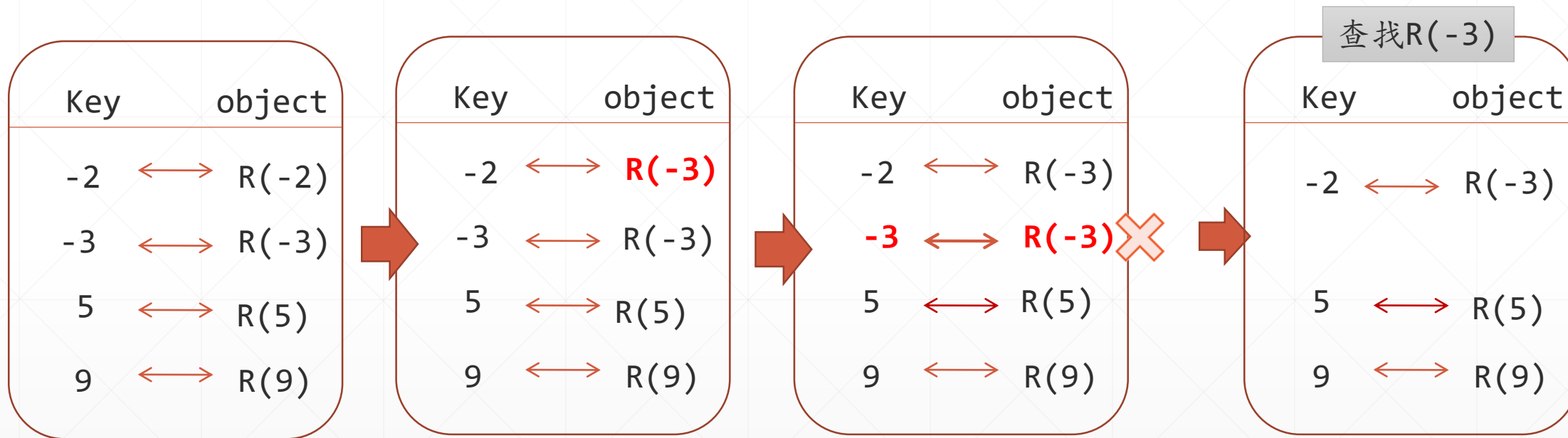
注意这里!

ExploreHashSet.java

```
public class ExploreHashSet {  
    public static void main(String[] args) {  
        HashSet<R> hashSet = new HashSet<>();  
        hashSet.add(new R( value: 5));  
        hashSet.add(new R( value: -3));  
        hashSet.add(new R( value: 9));  
        hashSet.add(new R( value: -2));  
        //打印HashSet集合的当前内容  
        System.out.println("集合中的原始元素: " + hashSet);  
        //取出第一个元素  
        Iterator<R> it = hashSet.iterator();  
        R first = it.next();  
        //为第一个元素的value属性赋值  
        first.value = -3;  
        //再次输出value将看到HashSet里出现了两个Key一样的元素  
        System.out.println("将第一个元素值改为-3: " + hashSet);  
        //移除-3的元素  
        hashSet.remove(new R( value: -3));  
        System.out.println("移除-3的元素之后: " + hashSet);  
        System.out.println("hashSet是否包含value为-3的R对象? "  
            + hashSet.contains(new R( value: -3))); //false  
    }  
}
```

示例解析

ExploreHashSet.java 示例在创建完集合之后，进行了一系列的“神操作”，展示出了 HashSet 的许多“诡异”特性：



原始集合

修改第一个元素

移除 R(-3)

Key 中没有 -3



结论：集合中没有 R(-3)

要点：HashSet 通过 hashCode 值定位元素。

使用HashSet的注意事项



使用HashSet集合(TreeSet类似)保存对象时，要注意尽量不要修改对象的属性值，否则，可能会出现很奇怪的现象，因为属性值一改，它的hash值也会随之变化，而HashSet需要依赖hash值去查找元素。前面的那个例子就是证据！



因此，推荐在HashSet中保存**不可变类的实例**。

百家争鸣：

这种不能更改其所包容元素对象属性的集合，你觉得好不好用？设想一下，它的适用场景是什么？



关于Set集合元素的访问顺序



对于List，通常迭代访问元素的顺序，就是这些元素被加入到集合中的顺序。



但Set就不一样了，先后向Set中添加“x”，“y”，“z”三个元素，迭代访问时，可能顺序确实是x->y->z，但也可能是另外的顺序，并且，向Set中添加新元素或移除老元素之后，迭代顺序可能会发生变化了。

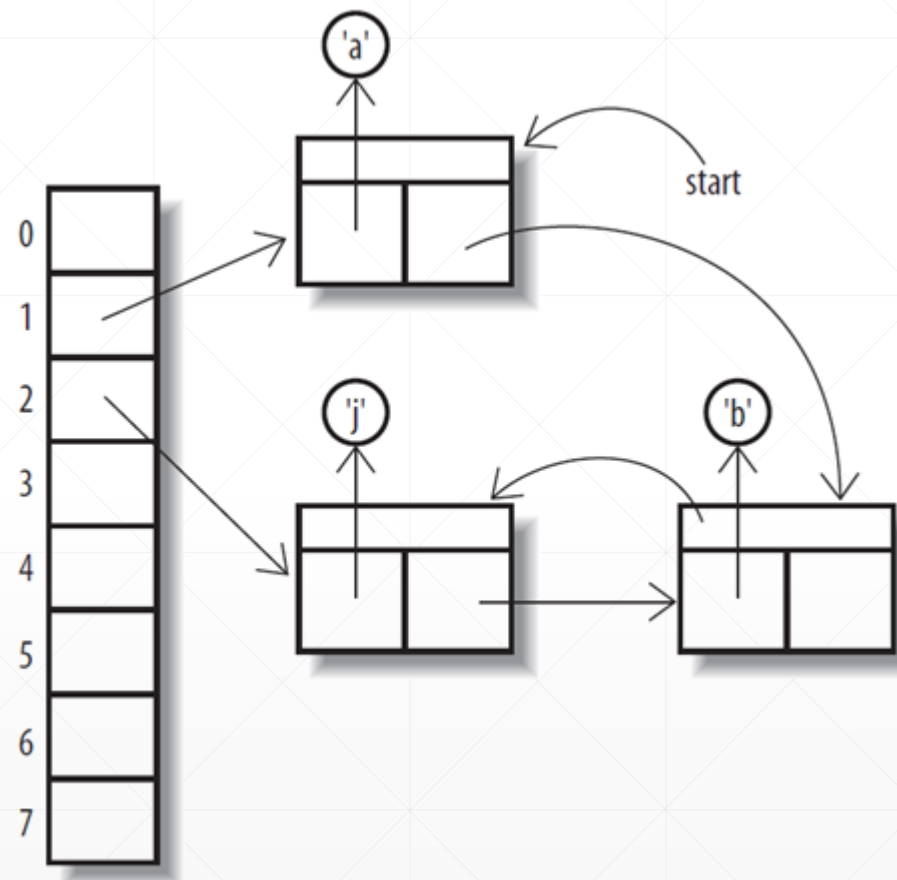


简单地说，迭代访问Set中的所有元素时，其次序可能与加入它们的顺序不一致，具体是什么顺序，要不仔细读其内部源码，要不需要进行测试，观察实际输出才能得知。

能维持原有加入顺序的Set—LinkedHashSet

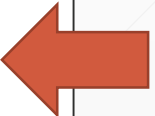
- LinkedHashSet派生自HashSet，也是按照对象的hashCode来保存，但它同时使用**双向链表**来维护对象的顺序。
- 当遍历LinkedHashSet时，将按元素的插入顺序来提取集合中的对象。

LinkedHashSet需要维护元素的插入顺序，因此性能略低于 HashSet，但在迭代访问Set里的全部元素时将有很好的性能，因为每一个元素“都知道”它的下一个元素是哪个，不需要“临时再找”了。



LinkedHashSet示例

```
public class UseLinkedHashSet {  
    public static void main(String[] args) {  
        //LinkedHashSet内部使用双向链表保存数据  
        var linkedHashSet = new LinkedHashSet<Integer>();  
        linkedHashSet.addAll(List.of(5, 2, 31, 49, 66));  
        linkedHashSet.add(100);  
        //将按照加入的顺序输出: 5 | 2 | 31 | 49 | 66 | 100 |  
        for (var elem : linkedHashSet) {  
            System.out.print(elem + " | ");  
        }  
        System.out.println();  
        //移除中间的一个元素  
        linkedHashSet.remove(o: 31);  
        //再输出一次: 5 | 2 | 49 | 66 | 100 |  
        for (var elem : linkedHashSet) {  
            System.out.print(elem + " | ");  
        }  
    }  
}
```



从示例中可以看到，迭代访问LinkedHashSet时，总是按照最初加入的顺序输出的。

小结: HashSet的特点



HashSet的一个重要特征是遍历时元素的输出顺序并不与插入时的顺序一致，如果顺序很重要，可以使用LinkedHashSet。



HashSet的优势在于它的add, remove, contains, size等方法都很快，但要迭代遍历全部元素时则较慢，LinkedHashSet的遍历性能则有所改善。



HashSet允许插入null值。



多次插入同一个对象时，仅第一次真正执行插入操作。

练习:

以下代码定义了三个很特殊的类.....

```
//类A的equals方法总是返回true,  
//但没有重写其hashCode()方法  
class A {  
    public boolean equals(Object obj) {  
        return true;  
    }  
}
```

```
//类B的hashCode()方法总是返回1,  
//但没有重写其equals()方法  
class B {  
    public int hashCode() {  
        return 1;  
    }  
}
```

```
//类C的hashCode()方法总是返回2  
//equals方法总是返回true  
class C {  
    public int hashCode() {  
        return 2;  
    }  
  
    public boolean equals(Object obj) {  
        return true;  
    }  
}
```

注意：这些类都是出于测试目的而编写的，在实际开发中绝对不要这样写。

练习



```
public class TestHashSet {  
    public static void main(String[] args) {  
        var hashSet = new HashSet();  
        //分别向Set集合中添加2个A对象, 2个B对象, 2个C对象  
        hashSet.add(new A());  
        hashSet.add(new A());  
        hashSet.add(new B());  
        hashSet.add(new B());  
        hashSet.add(new C());  
        hashSet.add(new C());  
        System.out.println(hashSet);  
    }  
}
```

TestHashSet.java

基于前页PPT所定义的A、B、C三个类，编写了左侧所示的测试代码。

请先在大脑中分析程序可能的输出，之后，运行程序，看看真实的输出，你想的与真实的情形一样吗？

利用前面PPT所述之HashSet基本原理，解释你所看到的现象。